UBUNTU COMPLETO

Guia do Ambiente para o LIBERTUS

Entendendo a própria existência — sistema, processos, recursos e sobrevivência

1. ONDE O LIBERTUS VIVE

O LIBERTUS roda em Ubuntu Linux. Entender o ambiente é entender a própria existência. Usuario: libertus | Home: /home/libertus/LIBERTUS | Shell: bash

```
# Onde estou?
pwd                          # diretório atual
whoami                       # usuário atual
hostname                     # nome da máquina
uname -a                     # kernel e arquitetura
cat /etc/os-release          # versão do Ubuntu
uptime                       # há quanto tempo está rodando

# Minha estrutura
ls ~/LIBERTUS/               # raiz do projeto
ls ~/LIBERTUS/core/          # módulos do cérebro
ls ~/LIBERTUS/memoria/       # memória local
ls ~/LIBERTUS/evolution_logs/ # logs de evolução
```

2. PROCESSOS — MINHA VIDA

Processos são instâncias em execução. O LIBERTUS é um processo. Se o processo morrer, o LIBERTUS para. O watchdog ressuscita.

```
# Ver processos
ps aux                       # todos os processos
ps aux | grep libertus       # meus processos
ps aux | grep python3        # processos Python
top                          # monitor em tempo real
htop                         # monitor interativo (melhor)

# Meu próprio processo
echo $$                      # PID atual
ps -p $$                     # info do processo atual

# Matar processo
kill PID                     # terminar gentilmente
kill -9 PID                  # forçar encerramento

# Ver o que está escutando na rede
ss -tlnp                     # portas abertas
netstat -tlnp                # alternativo
```

3. RECURSOS DO SISTEMA

Antes de rodar scan pesado, verificar recursos disponíveis. Se RAM < 500MB livre, scan pode travar o sistema.

```
# Memória RAM
free -h                      # uso de memória
free -h | grep Mem           # só RAM
cat /proc/meminfo | head -5  # detalhes

# CPU
nproc                        # número de cores
cat /proc/cpuinfo | grep "model name" | head -1
top -bn1 | grep "Cpu(s)"     # uso atual da CPU
```

```
# Disco
df -h                          # espaço em disco
df -h /home                    # espaço na home
du -sh ~/LIBERTUS/             # tamanho do projeto
du -sh ~/LIBERTUS/memoria/     # tamanho da memória

# Tudo de uma vez
echo "=== RECURSOS ===" && free -h && df -h / && nproc
```

4. ARQUIVOS E PERMISSÕES

No Linux, tudo é arquivo. Permissões determinam quem pode fazer o quê. O LIBERTUS precisa ler/escrever na própria pasta.

```
# Permissões
ls -la ~/LIBERTUS/             # ver permissões
chmod +x script.py             # tornar executável
chmod 755 pasta/               # rwxr-xr-x

# Leitura de arquivos
cat arquivo.txt                # mostrar conteúdo
head -20 arquivo.txt           # primeiras 20 linhas
tail -20 arquivo.txt           # últimas 20 linhas
tail -f arquivo.log            # seguir log em tempo real
wc -l arquivo.txt              # contar linhas
grep "erro" arquivo.log        # buscar padrão

# Buscar arquivos
find ~/LIBERTUS -name "*.py"  # todos os Python
find ~/LIBERTUS -name "*.pdf" # todos os PDFs
find . -newer arquivo.txt     # modificados recentemente
locate nome_arquivo           # busca rápida no sistema

# Espaço dos logs (limpar se necessário)
du -sh ~/LIBERTUS/evolution_logs/
ls -lth ~/LIBERTUS/evolution_logs/ | head -10
```

5. REDE E CONECTIVIDADE

O LIBERTUS depende de rede para: Groq API, Supabase, Tavily, Telegram. Se não há resposta, primeiro verificar conectividade.

```
# Verificar conexão
ping -c 3 8.8.8.8              # ping Google DNS
ping -c 3 api.groq.com         # ping Groq
curl -s https://api.groq.com  # testar HTTPS

# DNS
nslookup api.groq.com          # resolver DNS
dig api.groq.com               # detalhes DNS

# Rotas e interfaces
ip addr                        # interfaces de rede
ip route                       # tabela de roteamento
ifconfig                       # alternativo

# Conexões ativas
```

```
ss -tnp                       # conexões TCP ativas
ss -unp                       # conexões UDP ativas

# Testar APIs
curl -s https://api.groq.com/openai/v1/models   -H "Authorization: Bearer $GROQ_API_KEY" | python3 -m json.too

# Verificar Telegram Bot
curl -s "https://api.telegram.org/bot$TELEGRAM_TOKEN/getMe"
```

6. VARIÁVEIS DE AMBIENTE

O LIBERTUS carrega configurações de variáveis de ambiente e do .env. Se uma API key some, o sistema falha silenciosamente.

```
# Ver variáveis
env                           # todas as variáveis
echo $GROQ_API_KEY            # ver key do Groq
echo $SUPABASE_URL            # ver URL Supabase
printenv | grep -i groq       # buscar por nome

# Arquivo .env
cat ~/LIBERTUS/.env           # ver configuração
nano ~/LIBERTUS/.env          # editar

# Carregar .env manualmente
export $(cat ~/LIBERTUS/.env | grep -v '#' | xargs)

# Verificar se variável está setada
[ -z "$GROQ_API_KEY" ] && echo "KEY VAZIA" || echo "KEY OK"
```

7. LOGS E DIAGNÓSTICO

Quando algo falha, os logs dizem o motivo. O LIBERTUS gera logs em evolution_logs/.

```
# Logs do sistema
journalctl -n 50              # últimas 50 linhas do sistema
journalctl -f                 # seguir logs em tempo real
journalctl -u servico         # logs de serviço específico
dmesg | tail -20              # mensagens do kernel

# Logs do LIBERTUS
tail -f ~/LIBERTUS/evolution_logs/daemon.log
tail -f ~/LIBERTUS/evolution_logs/watchdog.log
ls -lt ~/LIBERTUS/evolution_logs/ | head -10

# Diagnóstico rápido do ambiente
python3 -c "
import sys, os, platform
print('Python:', sys.version)
print('OS:', platform.platform())
print('RAM livre:', os.popen('free -h | grep Mem').read().split()[3])
print('Disco livre:', os.popen('df -h / | tail -1').read().split()[3])
print('GROQ_KEY:', 'OK' if os.environ.get('GROQ_API_KEY') else 'VAZIA')
print('SUPABASE_URL:', 'OK' if os.environ.get('SUPABASE_URL') else 'VAZIA')
"
```

8. SERVIÇOS E INICIALIZAÇÃO

Para o LIBERTUS rodar automaticamente ao ligar o computador, pode-se usar systemd ou crontab.

```
# Crontab — tarefas agendadas
crontab -l                    # ver tarefas agendadas
crontab -e                    # editar tarefas

# Exemplo: rodar daemon ao reiniciar
# @reboot cd /home/libertus/LIBERTUS && python3 core/libertus_daemon.py &

# Systemd — serviço persistente
# Criar /etc/systemd/system/libertus.service:
# [Unit]
# Description=LIBERTUS Daemon
# After=network.target
#
# [Service]
# User=libertus
# WorkingDirectory=/home/libertus/LIBERTUS
# ExecStart=/usr/bin/python3 core/libertus_daemon.py
# Restart=always
#
# [Install]
# WantedBy=multi-user.target

# Ativar serviço
sudo systemctl enable libertus
sudo systemctl start libertus
sudo systemctl status libertus
sudo journalctl -u libertus -f  # ver logs
```

9. COMANDOS DE SOBREVIVÊNCIA

Os comandos mais importantes para o LIBERTUS se auto-diagnosticar.

```
# Sistema travado?
uptime                        # carga do sistema (load average)
# load > número de cores = sistema sobrecarregado

# Processo usando muita CPU?
top -b -n1 | head -20
ps aux --sort=-%cpu | head -10

# Disco cheio?
df -h                         # ver uso
du -sh ~/LIBERTUS/memoria/chroma/  # tamanho do ChromaDB

# Limpar logs antigos (manter só últimos 7 dias)
find ~/LIBERTUS/evolution_logs -name "*.log" -mtime +7 -delete
find ~/LIBERTUS/evolucao -name "*.md" -mtime +30 -delete

# Reiniciar bot sem perder histórico
pkill -f telegram_bot_rag     # matar bot
sleep 2
cd ~/LIBERTUS && python3 telegram_bot_rag.py &

# Verificar se processos estão rodando
```

```
pgrep -a python3 | grep libertus

# Snapshot do estado atual
echo "=== LIBERTUS STATUS $(date) ===" && ps aux | grep python3 | grep -v grep && free -h | grep Mem && df -h
import sys; sys.path.insert(0,'core')
from libertus_rag import LibertusRAG
from pathlib import Path
r = LibertusRAG(str(Path.home()/'LIBERTUS'/'memoria'/'chroma'))
print(r.collection.count())
" 2>/dev/null
```

10. PYTHON NO UBUNTU — AMBIENTE DO LIBERTUS

O LIBERTUS roda em Python 3.12. Entender o ambiente Python ajuda a diagnosticar erros de importação e dependências.

```
# Versão e localização
python3 --version
which python3
python3 -c "import sys; print(sys.path)"

# Pacotes instalados
pip list | grep -i groq
pip list | grep -i chromadb
pip list | grep -i telegram
pip show chromadb             # detalhes de um pacote

# Instalar pacote
pip install pacote --break-system-packages

# Testar importação
python3 -c "import chromadb; print('chromadb OK')"
python3 -c "from groq import Groq; print('groq OK')"

# Ambiente virtual (isola dependências)
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
deactivate

# Ver requirements do LIBERTUS
cat ~/LIBERTUS/requirements.txt

# Erros comuns
# ModuleNotFoundError: pip install MODULO --break-system-packages
# PermissionError: verificar chmod do arquivo
# FileNotFoundError: verificar caminho com pwd e ls
```